UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 08

Implementación en OmegaUP

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 Panorama de la sesión **2**

2 Solución — Problema 1: Separación pares e impares **3**
2.1 Solución de referencia 3
2.2 Errores frecuentes 3

3 Solución — Problema 2: Separación enumerando 1s **4**
3.1 Solución de referencia 4
3.2 Discrepancia enunciado vs. juez 4

4 Solución — Problema 3: Chasquidos Mágicos **5**
4.1 Solución de referencia 5
4.2 Errores frecuentes 5

5 Solución — Problema 4: Algoritmo de Phil **6**
5.1 Solución de referencia 6
5.2 Puntos clave de la implementación 6
5.3 Errores frecuentes 7

6 FAQs de la sesión **7**

Panorama de la sesión

Dinámica: equipos de 3, sesión de laboratorio frente a la computadora. El objetivo es someter los cuatro problemas de la sesión 07 a OmegaUP y obtener `AC` en todos.

Flujo sugerido:

0–5 min Abrir el cuaderno de Jupyter y OmegaUP en paralelo. Recordar el flujo: implementar en el cuaderno, verificar casos, someter.

5–45 min Trabajo en equipos. Los problemas 1 y 3 son los más rápidos; sugerir empezar por ellos para acumular puntos.

45–55 min Plenaria exprés: ¿quién obtuvo AC en cada problema? ¿Qué error fue el más común?

55–60 min Cierre: mostrar la solución del problema más difícil (Algoritmo de Phil) si ningún equipo lo terminó.

Orden de dificultad de implementación: Sep. pares/imp. $<$ Chasquidos $<$ Sep. enum. 1s $<$ Alg. de Phil.

Solución — Problema 1: Separación pares e impares

• Solución de referencia

```
#include <iostream>
using namespace std;
int main() {
    int n;
    while (cin >> n && n != 0) {
        cout << (n & 0xAAAAAAAA) << " " << (n & 0x55555555) << "\n";
    }
}
```

Dos operaciones AND con máscaras fijas. No hay bucle ni condicional.

• Errores frecuentes

- **Máscaras invertidas:** imprimir `0x55` primero y `0xAA` segundo. El enunciado pide a (posiciones impares = `0xAA`) primero.
- **Usar `int` con signo y n grande:** no es problema aquí porque $n < 2^{31} - 1$, pero conviene usar `unsigned int` o `long long` si hay dudas.
- **Error en el ejemplo del enunciado:** el enunciado original de OmegaUP muestra `5 2` para $n = 7$, pero los archivos del juez esperan `2 5`. La solución con máscaras produce `2 5`, que es la correcta para obtener AC.

Solución — Problema 2: Separación enumerando 1s

• Solución de referencia

```
#include <iostream>
using namespace std;
int main() {
    int n;
    while (cin >> n && n != 0) {
        int a = 0, b = 0, conteo = 0;
        for (int bit = 0; bit < 31; bit++) {
            if ((n >> bit) & 1) {
                conteo++;
                if (conteo % 2 == 1) a |= (1 << bit);
                else                 b |= (1 << bit);
            }
        }
        cout << a << " " << b << "\n";
    }
}
```

El bucle recorre los 31 bits inferiores de izquierda a derecha. Cada vez que encuentra un 1 incrementa el contador y lo asigna según paridad.

• Discrepancia enunciado vs. juez

El enunciado del OmegaUP describe el algoritmo de enumerar 1s, pero los archivos de casos del juez corresponden en realidad a la máscara `0x55`/`0xAA` (el complemento del problema 1).

Qué hacer en clase:

1. Pedir a los equipos que implementen el bucle de enumeración (como dice el enunciado) y sometan.
2. Si obtienen `WA`, pedirles que comparen sus respuestas con las del juez usando los casos de ejemplo.
3. Discutir la discrepancia: ¿el juez o el enunciado tiene la razón? El juez siempre gana — sus archivos de salida son la fuente de verdad.
4. Si el bucle da `WA`, probar con `a = n & 0x55555555; b = n & 0xAAAAAAAA`.

Esta situación es una lección práctica valiosa: en competencia, si tu lógica parece correcta pero el juez dice `WA`, buscas el caso que rompe tu solución — y a veces el enunciado tiene un error.

Solución — Problema 3: Chasquidos Mágicos

• Solución de referencia

```
#include <iostream>
using namespace std;
int main() {
    long long n, c;
    while (cin >> n >> c) {
        cout << (((c >> (n-1)) & 1) ? "PRENDIDA" : "APAGADA") << "\n";
    }
}
```

Una sola expresión de bits. El problema no especifica condición de terminación, así que se lee hasta fin de archivo.

• Errores frecuentes

- **`int` para c:** con c hasta 10^8 cabe en `int` (máx. $\approx 2.1 \times 10^9$), pero el desplazamiento `c » (n-1)` con n hasta 30 requiere que `c` sea al menos de 32 bits. Usar `long long` elimina cualquier duda.
- **Simular con bucle:** algunos equipos intentan simular el estado de cada enchufe en cada chasquido. Con $c = 10^8$ eso es TLE inmediato. Guiarlos a la fórmula.
- **Condición de terminación:** el enunciado da varios pares de entrada sin indicar cuántos hay. La lectura debe ser `while (cin » n » c)`.

Solución — Problema 4: Algoritmo de Phil

• Solución de referencia

```
#include <iostream>
#include <string>
using namespace std;

long long algoritmoPbil(string A) {
    const long long MOD = 1000000007LL;
    long long S = 0;
    while (!A.empty()) {
        int n = A.size();
        if (n % 2 == 1) {
            int mid = n / 2;
            S = (S * 2 + (A[mid] - '0')) % MOD;
            A.erase(mid, 1);
        } else {
            int izq = n / 2 - 1, der = n / 2;
            int b1 = A[der] - '0', b2 = A[izq] - '0'; // mayor, menor
            if (b2 > b1) swap(b1, b2);
            S = (S * 4 + b1 * 2 + b2) % MOD;
            A.erase(der, 1);   // primero el de mayor índice
            A.erase(izq, 1);
        }
    }
    return S;
}

int main() {
    int k; cin >> k;
    for (int i = 1; i <= k; i++) {
        string A; cin >> A;
        cout << "Caso #" << i << ": " << algoritmoPbil(A) << "\n";
    }
}
```

• Puntos clave de la implementación

- **Orden de erase:** en el caso par, borrar primero el índice mayor (`der`) y luego el menor (`izq`). Si se borra `izq` primero, el índice `der` se desplaza y se borra el carácter equivocado.
- **Acumulación de S:** el caso impar multiplica por 2 (se agrega un bit); el caso par multiplica por 4 (se agregan dos bits). Aplicar módulo en cada paso evita

desbordamiento.

- **Mayor/menor:** en caso de empate, el orden no importa (ambos bits son iguales), así que cualquier orden produce el mismo S.
- **Complejidad:** `string::erase` es $O(n)$; el bucle hace $O(\log n)$ iteraciones de longitud decreciente a la mitad. Total: $O(n \log n)$ para cadenas de hasta 500 caracteres — sin problema.

- **Errores frecuentes**

- **Acumular S como cadena:** concatenar caracteres y convertir al final desborda rápidamente. La acumulación numérica con módulo es la forma correcta.
- **Olvidar el módulo:** con cadenas de 500 bits, S puede exceder 10^9+7; aplicar módulo solo al final produce `WA`.
- **Índices del centro:** confundir `n/2` (centro derecho) con `n/2 - 1` (centro izquierdo) en el caso par.

FAQs de la sesión

Los tres culpables más comunes en esta sesión:

1. **Formato de salida:** un espacio extra, una línea en blanco adicional o mayúsculas incorrectas (`prendida` en lugar de `PRENDIDA`).
2. **Condición de terminación incorrecta:** algunos problemas terminan con 0; otros, con fin de archivo. Mezclarlos da `WA` o cuelga el programa.
3. **Tipo de dato insuficiente:** usar `int` donde se necesita `long long`.

Estrategia estándar:

1. Prueba con los ejemplos del enunciado — deben dar exactamente la salida esperada, carácter por carácter.
2. Prueba casos extremos: $n = 0$, n muy grande, todos los bits en 1, todos en 0.
3. Si sospechas de un caso específico, pruébalo en el cuaderno de Jupyter antes de someter de nuevo.